

# Aprendizado por Reforço no jogo digital Asteroids

João Victor Evangelista Cruz  
Departamento de Ciência da Computação  
Universidade Federal de Minas Gerais  
Belo Horizonte, Brasil  
joaovecruz@ufmg.br

**Abstract**—This work presents the development and validation of a Reinforcement Learning (RL) agent applied to navigation and combat control in a custom C++ implementation of the classic game Asteroids. Built natively using the SDL library, the game engine was integrated with a Python training pipeline using Pybind11 and Gymnasium, modeling the environment as a Markov Decision Process (MDP). We propose an efficient state engineering technique based on a context window that tracks only the two closest asteroids to the spaceship, mapping the data into a normalized vector of 21 float elements. Using the Proximal Policy Optimization (PPO) algorithm in an Actor-Critic architecture, the agent was evaluated under various reward shaping schemes. Experimental results in scenarios with 10 asteroids showed that increasing the collision penalty from  $-10$  to  $-20$  promoted robust defensive behaviors, achieving stable survival and firing accuracy above 80%. Mathematical convergence was demonstrated by the explained variance consolidating near 0.9 and the entropy loss decreasing to  $-0.6$ , confirming the transition from exploratory actions to a highly deterministic control policy. We conclude that structured state representations based on spatial proximity are highly viable and sample-efficient for learning in continuous physics environments, bypassing the high computational overhead of raw pixel-based visual processing approaches.

**Index Terms**—Reinforcement Learning; Proximal Policy Optimization; Feature Engineering; Asteroids; C++.

## I. INTRODUÇÃO E CONTEXTO

O Aprendizado por Reforço (RL - Reinforcement Learning) é uma vertente da Inteligência Artificial que funciona por decisão sequencial. Agentes autônomos aprendem a tomar decisões ao interagir com um ambiente. Diferentemente de outros tipos de aprendizado, como o supervisionado e o não supervisionado, o agente não recebe dados rotulados ou extrai informações ocultas nos dados. Em vez disso, aprende por tentativa e erro, recebendo uma recompensa por uma ação realizada em um determinado estado, e após milhares de tentativas, o agente aprende qual ação escolher em cada situação para maximizar a recompensa recebida.

No estudo do Aprendizado por Reforço, os jogos digitais funcionam como ótimos laboratórios de testes e aprendizado. Eles oferecem regras bem definidas e sistemas de recompensas claros, mas com a possibilidade de ter ambientes altamente estocásticos, caóticos e de alta dimensionalidade, e assim, oferecer desafios a serem resolvidos.

Portanto, integrando as áreas de Aprendizado por Reforço e Desenvolvimento de Jogos Digitais, este trabalho tem a proposta de desenvolver um agente através de Aprendizado por Reforço para aprender a jogar um jogo inspirado no Asteroids, do Atari, desenvolvido em C++.

O algoritmo de Aprendizado por Reforço utilizado no trabalho é o PPO (Proximal Policy Optimization). Ele tem uma arquitetura Actor-Critic, dividindo o aprendizado em duas redes neurais: o Ator, que escolhe as ações do agente (como girar ou atirar), e o Crítico, que avalia a qualidade dessas escolhas para prever as recompensas futuras. O algoritmo tem uma função de objetivo truncada (Clipped Objective), que limita o tamanho das atualizações do Ator para garantir que ele mude a sua política de forma estável e segura, sem desorganizar os comportamentos benéficos já consolidados.

## A. Objetivos

Para o desenvolvimento do trabalho, foram definidos os seguintes objetivos:

- Adaptar o jogo de forma a possibilitar o treinamento do agente: O jogo, originalmente feito para jogabilidade humana através das teclas do computador, tem que ser modificado para funcionar em estados, para permitir que o agente escolha uma ação, essa ação seja executada e uma recompensa seja retornada, em uma modelagem conhecida como Processo de Decisão de Markov (MDP).
- Integrar o jogo com o ambiente de treinamento: É necessário um mecanismo que permita a comunicação entre o jogo, feito em C++ e o pipeline de treinamento, executado em Python.
- Treinar um modelo de Machine Learning para aprender a jogar o jogo: Para treinar um modelo, é necessário definir seus hiperparâmetros e alguns outros parâmetros referentes aos estados do jogo e às recompensas, e testar o comportamento do agente, de forma a possibilitar e otimizar o treinamento.

## B. Estrutura

Este documento está organizado em capítulos conforme a seguinte estrutura:

O Capítulo 2 apresenta o Referencial Teórico e o mapeamento dos trabalhos correlatos na literatura de Aprendizado por Reforço aplicado a jogos.

O Capítulo 3 detalha a metodologia aplicada, descrevendo a arquitetura do sistema, a engenharia do espaço de observações e as funções de recompensa implementadas.

O Capítulo 4 expõe e discute os resultados experimentais obtidos nos treinamentos.

O Capítulo 5 apresenta as conclusões alcançadas e propõe direcionamentos para trabalhos futuros, seguido pela declaração de disponibilidade de dados no repositório público.

Por fim, a parte final mostra as referências bibliográficas utilizadas no presente trabalho.

## II. TRABALHOS CORRELATOS

O marco fundamental para a aplicação de aprendizado por reforço profundo em ambientes complexos foi estabelecido pelo trabalho [1] realizado pela Deepmind - principal divisão de inteligência artificial do Google atualmente. Os autores desenvolveram um algoritmo, denominado Deep Q-Network (DQN), capaz de aprender políticas diretamente a partir de entradas de alta dimensionalidade. O modelo foi validado em 26 jogos clássicos do Atari 2600, incluindo o próprio Asteroids, recebendo apenas os pixels brutos e a pontuação do jogo como entrada, através de Redes Neurais Convolucionais (CNNs). Os resultados mostraram um desempenho próximo ou em alguns casos superior ao de jogadores humanos. Em contraste com a abordagem off-policy e baseada em valor do DQN, e a visualização do jogo através de pixels, o presente trabalho adota o algoritmo Proximal Policy Optimization (PPO), explorando uma otimização direta baseada em gradiente de política, usando uma observação do jogo baseada nos dados dos objetos enviados pelo jogo ao ambiente de treinamento.

Embora ambientes clássicos tenham sido importantes para o avanço da área do Aprendizado por Reforço, os estudos demonstram que os agentes muitas vezes buscam memorizar estratégias específicas para ambientes determinísticos em vez de generalizar o aprendizado. A partir disso, os pesquisadores da OpenAI desenvolveram o Procgen Benchmark [2], um conjunto de 16 ambientes com geração procedural para testar o aprendizado por reforço em diferentes ambientes estocásticos. Um desses ambientes, o CaveFlyer, exige o controle de uma nave cujos movimentos e mecânica foram inspirados no jogo Asteroids. De maneira parecida, o presente trabalho tem por base investigar a aplicação do algoritmo PPO em um ambiente de geração procedural. Este, porém, utiliza uma versão customizada em C++, empregando um Perceptron Multicamadas (MLP) para processar as features do jogo, em vez de processamento de imagens com CNNs.

Além disso, os ambientes clássicos serviram como laboratório para que projetos mais desafiadores pudessem ser desenvolvidos. Projetos desenvolvidos pela OpenAI [3] e pela Deepmind [4], são projetos que exploram a escalabilidade de agentes em ambientes de estratégia em tempo real. O primeiro utilizou o jogo Dota 2 e treinou o OpenAI Five, um sistema multiagente que controla os cinco heróis do time e foi capaz de derrotar os campeões mundiais de Dota 2 da época. Já no segundo projeto, foi desenvolvido o Alphastar, um agente que funciona no jogo StarCraft II conseguiu atingir o nível mais alto do servidor e superou 99.8% dos jogadores humanos. Assim como o presente trabalho, estes também utilizam uma versão do algoritmo PPO e usam as features do jogo diretamente, extraindo-as de uma API dos dados dos jogos.

## III. METODOLOGIA

Para o desenvolvimento do trabalho, a criação do jogo e do ambiente de treinamento, assim como o treinamento do agente se deram da seguinte forma:

### A. O jogo Asteroids

Para esse trabalho, foi utilizado uma versão do jogo Asteroids, feito na linguagem C++ utilizando a biblioteca SDL (Simple DirectMedia Layer), bastante utilizada no desenvolvimento de jogos digitais.

O jogo consiste em uma nave no espaço sideral, que é controlada pelo player e asteroides que surgem no início do jogo. A nave pode fazer os seguintes movimentos: Girar para a esquerda, girar para a direita, acelerar e atirar. Esse tiro possui um tempo de espera de 1 segundo entre um tiro e outro. No jogo, existe a característica da inércia, de forma que quando a nave para de acelerar, o movimento continua durante um tempo, ela não para imediatamente. A nave sempre surge no início do jogo no centro da tela, e os asteroides surgem em posições aleatórias, e cada um possui uma única direção e viajam nessa direção com uma velocidade constante até o final do jogo ou até serem destruídos.

O objetivo do jogo é destruir todos os asteroides sem deixar que eles encostem na nave. Caso eles se choquem com a nave, a nave é destruída e é fim de jogo. Além disso, há no jogo uma topologia toroidal, ou seja, os objetos conseguem ultrapassar as bordas da tela, e ao fazerem isso, surgem na borda do lado contrário da tela.



Fig. 1. Interface gráfica do jogo Asteroids feita em C++.

### B. Transformação do jogo para o treinamento

O jogo foi desenvolvido inicialmente como um jogo para ser jogado por humanos, usando o teclado do computador. Porém, para treinar um agente via Aprendizado por Reforço, é necessário que o ambiente esteja modelado por um modelo conhecido como MDP - Processo de Decisão de Markov.

O MDP é um modelo matemático para resolver problemas de decisão sequencial. Ele molda o ambiente de forma a ter

estados, que são as situações possíveis em que o agente pode se encontrar e para cada situação dessa, o agente escolhe uma ação a ser executada. Após essa ação, o agente recebe uma recompensa, que funciona como um feedback, que pode ser positivo ou negativo, e serve para mostrar ao agente se aquela ação ou sequência de ações foram boas ou não naqueles estados. Após isso, o agente vai para o estado seguinte e isso se repete, de forma que o agente aprende quais ações em quais situações maximizam as recompensas.

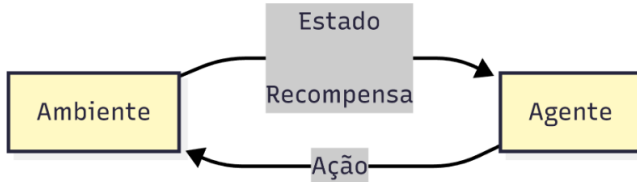


Fig. 2. Ilustração do Processo de Decisão de Markov.

Portanto, o jogo foi modificado de forma que, em vez de funcionar com interação humana, ele funciona através de duas funções principais: A função Reset, que é chamada no início do treinamento e ao final de cada episódio, e que retorna o estado de observações inicial do jogo; E a função Step, que recebe como parâmetro a ação a ser executada no estado atual, e essa ação é executada por 4 frames do jogo, e então é calculada a recompensa daquele estado de acordo com o que aconteceu nesses 4 frames, e a função retorna a recompensa, o estado de observações do próximo estado e a informação se o episódio terminou. Os detalhes sobre o espaço de observações se encontram na seção 3.4.

Um episódio termina quando o agente vence o jogo, ao destruir todos os asteroides, quando ele perde o jogo, ao chocar a nave com um asteroide, ou quando o episódio atinge a quantidade máxima de passos, definida como 6000 passos.

### C. Comunicação entre o jogo e o agente

A partir do jogo modelado como um MDP, foi necessário desenvolver a comunicação entre o jogo e o agente. Para isso, foi utilizada a biblioteca Pybind, que permite que o jogo possa ser construído como um módulo de extensão compilado para Python, permitindo que o ambiente de treinamento consiga importar o jogo como um módulo no Python e chamar as funções citadas anteriormente.

Além disso, é utilizada também a biblioteca Gymnasium, que tem como função modelar o ambiente de treinamento com as informações do espaço de ações e do espaço de observações, e assim, padronizar o ambiente para permitir o acoplamento de algoritmos, como da biblioteca Stable Baselines 3, que foi o algoritmo escolhido para o projeto. A figura 3 ilustra a comunicação entre as partes.

### D. Espaço de ações e de observações

Uma das principais partes do desenvolvimento do trabalho foi a definição do espaço de ações e do espaço de observações. Eles moldam como o agente visualiza o ambiente e o que

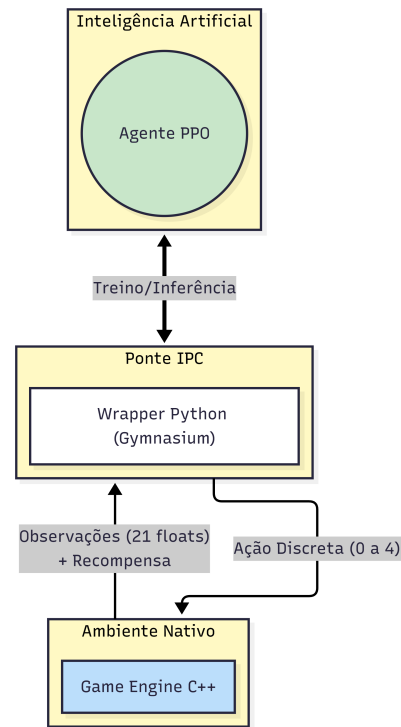


Fig. 3. Ilustração da comunicação entre agente e jogo.

ele pode fazer em cada situação. Para isso, foi utilizada a estratégia de janela de contexto, na qual, em vez de receber as informações de todos os asteroides que estão vivos no momento, ele recebe as informações de apenas os dois asteroides mais próximos da nave. Isso é importante, pois otimiza o treinamento, de forma a diminuir a quantidade de informações a serem processadas pelo agente. Essa modelagem não atrapalha o agente de tomar as decisões, pois os asteroides mais perigosos para o agente são os que estão mais próximos dele, então ele deve focar nesses asteroides antes de se preocupar com asteroides mais afastados.

A partir disso, os espaços foram modelados da seguinte forma:

**Espaço de ações:** Especifica as ações possíveis de serem realizadas. Para esse jogo, foi definido um espaço discreto, de tamanho 5, onde cada valor representa uma ação diferente:

- 0: Virar a nave para a direita
- 1: Virar a nave para a esquerda
- 2: Atirar
- 3: Acelerar
- 4: Não fazer nada

**Espaço de observações:** Especifica como o agente “enxerga” o jogo, as características de um determinado estado para que o agente possa tomar a decisão de qual ação escolher. Foi definido um espaço com 21 valores do tipo float, normalizados de forma a terem valores entre -1 e 1, com as seguintes informações da nave e dos dois asteroides mais próximos:

- Nave: Direção para qual a nave está apontando, velocidade da nave e cooldown atual do tiro da nave.

- Asteroides: Posição relativa do asteroide em relação a nave, distância entre o asteroide e a nave, produtos vetorial e cruzado entre os vetores da nave e do asteroide, além do tamanho e velocidade do asteroide.

A escolha de utilizar a posição relativa do asteroide em vez da absoluta se dá pela topologia toroidal do ambiente, para que a variação desses valores seja mais suave para o agente. Além disso, os produtos vetorial e cruzado entre os vetores da nave e do asteroide são fundamentais para que o agente saiba se o asteroide está ou não à frente da nave, e para qual lado ele deve virar para alinhar com o asteroide, facilitando a mecânica de mira.

#### E. Recompensas e Hiperparâmetros

As recompensas que o agente recebia foram definidas da seguinte maneira:

- +1 para cada asteroide destruído;
- +10 ao vencer o jogo (destruir todos os asteroides);
- -20 ao perder o jogo;
- -0.05 para cada tiro errado;

Dessa forma, com um alto peso na derrota, o agente entende que sobreviver é mais importante do que destruir os asteroides a qualquer custo. Além disso, a recompensa de tiros errados evita que o agente simplesmente atire para qualquer lado aleatoriamente, e faz com que ele mire antes de atirar.

Já os hiperparâmetros do PPO foram definidos da seguinte forma:

- Taxa de aprendizado(learning\_rate): 0,0003;
- Número de passos coletados antes de atualizar o modelo (n\_steps): 2048;
- Tamanho do lote(batch\_size): 256;
- Gamma: 0.99;
- Lambda: 0.95;
- Coeficiente de entropia(ent\_coef): 0.005;
- Tamanho da rede: Duas camadas ocultas de 128 neurônios cada;

São hiperparâmetros próximos do padrão do PPO, com algumas alterações como o aumento do tamanho do lote e do tamanho da rede, permitindo um aprendizado melhor.

#### IV. RESULTADOS EXPERIMENTAIS

Diversos testes foram realizados alterando-se o espaço de observações, as recompensas e funcionamento do jogo até chegar nas configurações descritas nas seções acima.

O planejamento das experimentações foi fazer testes em ambientes mais simples e depois aumentar a complexidade. Então, inicialmente, foi testada a configuração com a janela de contexto de tamanho 1 e recompensa de morte de -10 e funcionou bem para ambientes com 1 e 2 asteroides, o modelo conseguiu convergir bem e aprendeu a mirar no asteroide da janela e atirar quando estiver alinhado, obtendo boas recompensas, como mostram os gráficos das figuras 4 e 5, onde eles convergiram com cerca de 200.000 a 400.000 passos treinados.

Porém, ao fazer o treinamento com 3 asteroides, o modelo oscilou bastante e até conseguiu acertar os asteroides

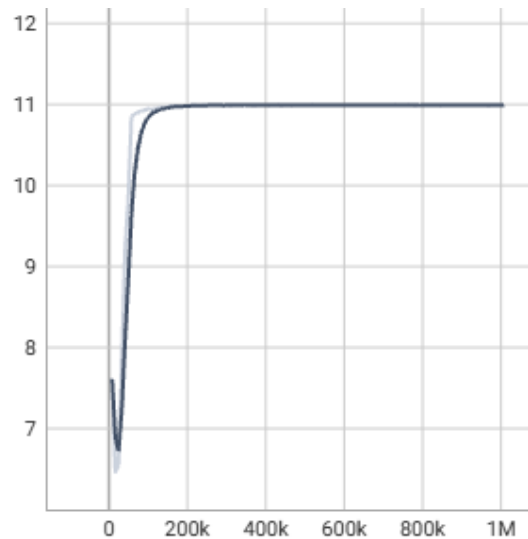


Fig. 4. Gráfico (Recompensas/Quantidade de passos) do ambiente com 1 asteroide.

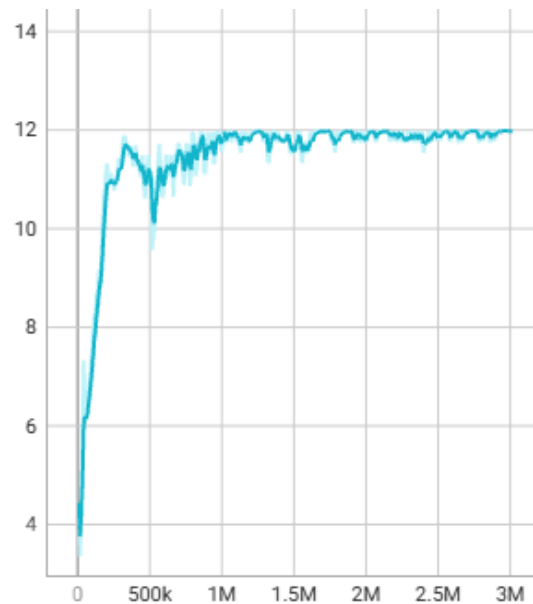


Fig. 5. Gráfico (Recompensas/Quantidade de passos) do ambiente com 2 asteroides.

algumas vezes, mas errava muitos tiros, e portanto teve uma recompensa muito oscilante, mesmo treinando até 8.000.000 de passo, como mostra a figura 6.

A partir disso, a janela de contexto foi aumentada para 2 asteroides, e foi feito um teste com o ambiente tendo 4 asteroides, para testar o comportamento do modelo com uma visualização mais ampla do ambiente. Os testes mostraram que o aumento da janela foi benéfica e que, ao conseguir “enxergar” mais asteroides próximos, o agente pôde entender melhor o ambiente e traçar uma estratégia robusta, na qual conseguiu mirar e destruir os asteroides de forma satisfatória, como pode ser observado na figura 7.

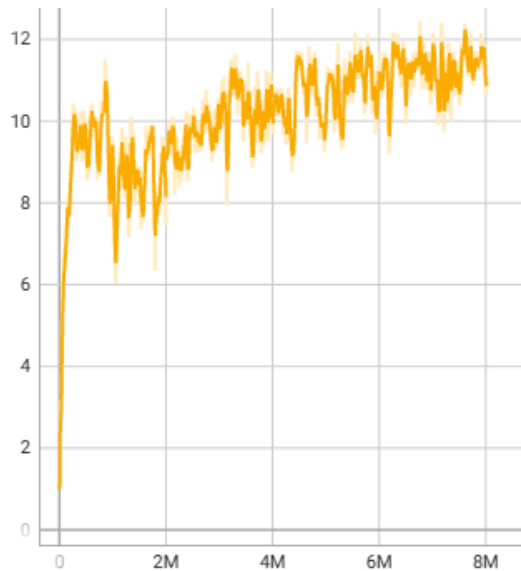


Fig. 6. Gráfico (Recompensas/Quantidade de passos) do ambiente com 3 asteroides.

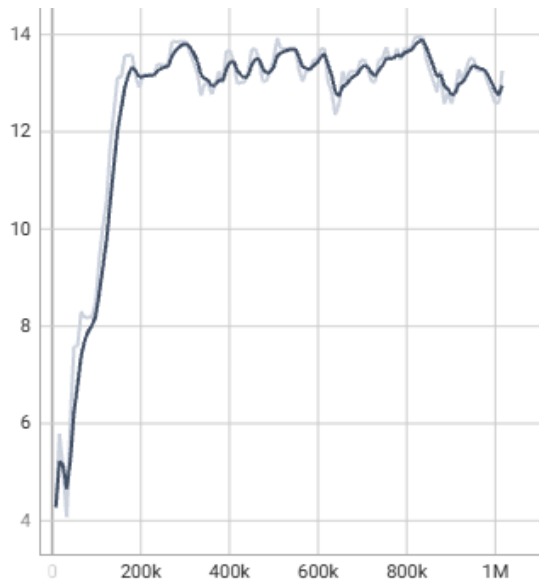


Fig. 7. Gráfico (Recompensas/Quantidade de passos) do ambiente com 4 asteroides.

Com esse resultado, a quantidade de asteroides no jogo foi aumentada para 10 asteroides, que é a quantidade alvo do trabalho, para testar se as configurações definidas já eram suficientes para que o modelo funcionasse com a versão final do jogo. Os testes mostraram que o modelo conseguiu aprender a mirar e atirar nos asteroides, mas que por ter mais asteroides, a recompensa recebida pela morte não era tão grande comparada com a recompensa recebida por destruir os asteroides, então, o agente entendia que valia a pena se aproximar bastante dos asteroides para destruí-los, já que isso tornava o acerto dos tiros mais fácil, mesmo que isso causava a morte da nave em alguns momentos.

Portanto, foi necessário modificar a recompensa de morte de -10 para -20, para dar mais peso à morte, para que o agente pudesse entender que é melhor errar alguns tiros e ficar vivo, do que acertar todos os tiros e acabar morrendo por causa disso. Então foi treinado um modelo com essas configurações e foram obtidos ótimos resultados: A nave aprendeu bem a mirar e atirar nos asteroides presentes na janela de contexto, e aprendeu a ficar a uma distância segura dos asteroides, tendo êxito em limpar a tela e vencer o jogo, destruindo os asteroides mais próximos um a um e errando poucos tiros, com uma acurácia acima de 80%, como mostra as figuras 8 e 9.

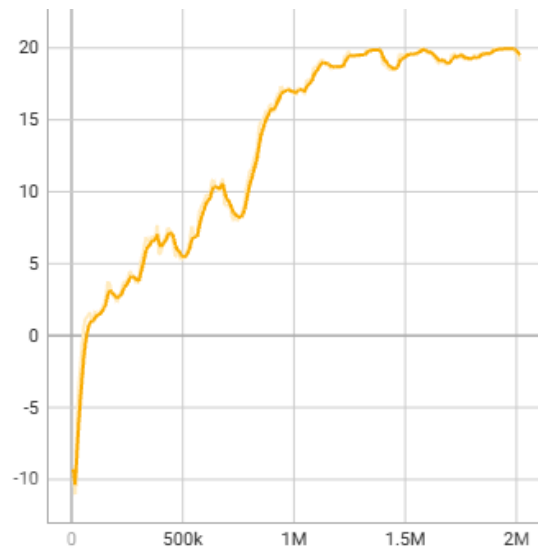


Fig. 8. Gráfico (Recompensas/Quantidade de passos) do ambiente com 10 asteroides.

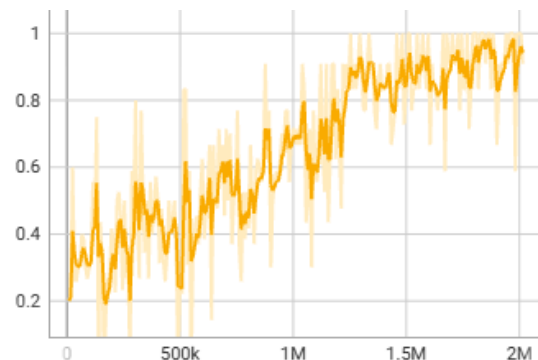


Fig. 9. Gráfico acurácia dos tiros no ambiente com 10 asteroides.

Além disso, as informações de perda de entropia (*entropy\_loss*) e variância explicada (*explained\_variance*), nas figuras 10 e 11, mostram que as ações escolhidas pelo modelo possuíam uma alta explicabilidade, ou seja, o agente sabia o que estava fazendo, tomava as decisões com base na política aprendida. Com isso, as ações, que no início eram bastante aleatórias, com o objetivo de exploração, se tornaram mais determinísticas, convergindo para a estratégia aprendida.

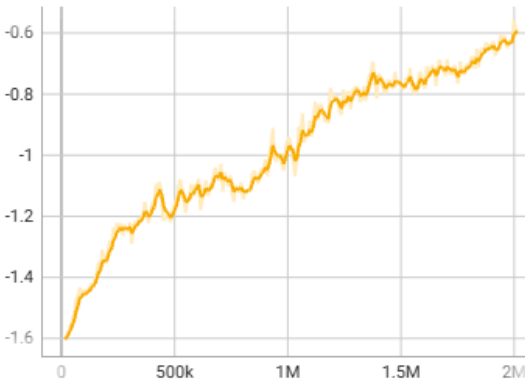


Fig. 10. Gráfico (Perda de entropia/Quantidade de passos) do ambiente com 10 asteroides.

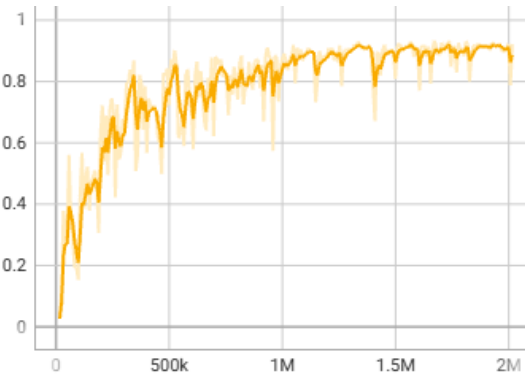


Fig. 11. Gráfico (Variância explicada/Quantidade de passos) no ambiente com 10 asteroides.

## V. CONCLUSÕES E TRABALHOS FUTUROS

Este trabalho apresentou o desenvolvimento e a validação de um agente de Aprendizagem por Reforço aplicado à navegação e combate em uma versão do jogo clássico Asteroids. O principal desafio se deu na modelagem de um espaço de observações capaz de representar de forma robusta e consistente um ambiente dinâmico, estocástico e com topologia toroidal, sem depender do processamento computacionalmente caro de imagens brutas através de redes convolucionais.

Os resultados experimentais confirmaram que a convergência do modelo junto ao algoritmo PPO está fortemente ligada à consistência da representação dos estados passados ao agente. Ao incluir dados como o produto vetorial e produto cruzado dos objetos, e utilizar e ajustar a estratégia de janela de contexto, o modelo demonstrou um ótimo desempenho ao construir uma política robusta. Essa estabilidade pode ser percebida pela rápida convergência da variância explicada para valores próximos de 0.9 e pela redução da entropia do sistema para -0.6, mostrando a transição de um comportamento puramente exploratório para uma política determinística e coordenada de sobrevivência e acurácia nos tiros.

Como limitações do projeto, destaca-se a simplificação do ambiente para um número fixo de asteroides por episódio, em vez de funcionar de forma parecida com o jogo original, na

qual quando os asteroides eram destruídos, outros menores surgiam no lugar. Foram feitos testes com as configurações finais do projeto em um ambiente onde a dinâmica de novos asteroides surgindo existia, e algo curioso aconteceu: Treinando um agente do zero para esse ambiente, o agente não conseguiu convergir para uma política adequada, porém ao aplicar o agente treinado com o ambiente de número fixo de asteroides, nesse novo ambiente com a dinâmica de novos asteroides, o agente teve um bom desempenho, limpando a tela na maior parte das vezes, porém morrendo algumas vezes por não estar acostumado com o surgimento desses novos asteroides.

A hipótese acerca desse resultado no treinamento do novo ambiente é de que, como há mais asteroides na tela, o agente acerta muitos asteroides que não estão na janela de contexto, e portanto ele ganha recompensas sem conseguir entender o que causou esse ganho, e assim, não consegue descobrir uma boa política para jogar o jogo.

Diante disso, propõe-se para trabalhos futuros a investigação desse novo ambiente, e a experimentação de novas ideias para esse desafio, como a remodelagem das recompensas, ou o aumento da janela de contexto. Além disso, sugere-se estudos investigando a utilização de outros algoritmos de aprendizado por reforço ou de outras formas de representação do estado de observação nesse mesmo ambiente construído, para a comparação dos resultados.

### A. Disponibilidade de Dados e Código

O código-fonte associado a esta pesquisa, incluindo as implementações nativas em C++, os cenários procedurais simulados e o ambiente de treinamento do agente está hospedado publicamente no GitHub sob a licença de código aberto MIT, disponível em: <https://github.com/jao-07/RL-Asteroids>.

### B. Declaração de Uso de Ferramentas de IA Generativa

Em conformidade com as diretrizes de integridade científica e ética em pesquisa da instituição, declara-se que ferramentas de Inteligência Artificial Generativa baseadas em Grandes Modelos de Linguagem (LLMs), foram utilizadas como assistentes técnicos no desenvolvimento deste trabalho. O uso da ferramenta limitou-se ao suporte de programação (co-piloto de código), incluindo a refatoração e otimização de algoritmos de gerência de memória em C++ (alinhamento estável do vetor de observações) e auxílio na detecção de falhas de segmentação (null pointer dereferences). Adicionalmente, a IA foi empregada no suporte à revisão gramatical e estruturação técnica da redação deste documento em língua portuguesa. Declara-se, contudo, que toda a formulação conceitual do MDP, análise de resultados, interpretação física dos hiperparâmetros de treinamento e redação final das seções permanecem sob responsabilidade intelectual exclusiva e autoria integral do aluno.

## REFERENCES

- [1] V. Mnih et al. Human-level control through deep reinforcement learning. Nature, Londres, vol. 518, n. 7540, pp. 529-533, February 2015.

- [2] K. Cobbe, C. Hesse, J. Hilton, and J. Schulman, "Leveraging Procedural Generation to Benchmark Reinforcement Learning," in Proceedings of the 37th International Conference on Machine Learning, 2020, pp. 2048–2056. Available: <https://proceedings.mlr.press/v119/cobbe20a.html>.
- [3] OpenAI et al., "Dota 2 with Large Scale Deep Reinforcement Learning," OpenAI, December 2019. [Online] Available: <https://openai.com/bibtex/openai2019dota.bib>.
- [4] O. Vinyals et al., Grandmaster level in StarCraft II using multi-agent reinforcement learning. Nature, Londres, vol. 575, n. 7782, pp. 350-354, October 2019.